

1 Methodology

The proposed vacuum grasping algorithm evaluates point cloud data to determine the optimal Target Center Point (TCP) and approach vector for a multi-pad vacuum gripper. The method ensures physical feasibility and maximizes grasp stability. The pipeline consists of four main phases, as illustrated in Figure 1.

1. **Point Cloud Preprocessing and Surface Discovery:** The raw point cloud undergoes preprocessing, including voxel downsampling and dynamic radius-based outlier removal, which preserves geometric features such as sharp edges and flat surfaces. To identify viable contact points and resolve normal vector ambiguities, an inward raycasting strategy is applied. The algorithm uses a configurable pattern, combining orthogonal (top-down) rays to thoroughly explore planar upper surfaces and spherical rays to capture all possible approach directions. The dot product between the ray direction and the estimated surface normal ensures that all vectors point consistently outward from the object.
2. **Pose Generation and Clearance Verification:** Valid ray hits are snapped to the nearest physical surface points using a KD-Tree search. For each point, the gripper’s Z-axis is aligned with the surface normal to establish a base approach pose. Before generating additional orientations, the algorithm performs a local clearance check within a defined safety volume. This volume is expanded by a *collision margin* (e.g., 5%) to absorb sensor noise and calibration inaccuracies. Furthermore, an *approach distance* is established along the normal vector to guarantee a safe linear trajectory prior to physical contact. Base poses that pass this collision filter are rotated around their local Z-axis in discrete angular steps to generate a set of multi-axis orientation candidates. To optimize computational efficiency, the total rotation range adapts to the hardware’s physical symmetry. For instance, the algorithm evaluates a 180° range for bilateral two-pad grippers, 90° for orthogonal four-pad configurations, and restricts rotation entirely (0°) for a single, radially symmetric circular cup.
3. **Candidate Evaluation (GSS Loop):** Each generated candidate undergoes evaluation to compute a Grasp Stability Score (GSS). This score assesses the physical viability of the grasp based on three criteria:
 - *Verticality Evaluation:* Assesses the tilt of the gripper’s approach vector. Poses deviating from the gravity vector (world Z-axis) are penalized to favor top-down picking. Grasps executed from below (upside-down) are heavily penalized using a decay factor (0.6) to account for gravity acting directly against the suction holding force.
 - *Torque Evaluation:* Calculates the planar distance (in the XY plane) from the proposed grasp point to the object’s estimated Center of Mass (CoM). Candidates with a smaller XY offset to the CoM receive higher scores, reducing induced torque during lifting.
 - *Multi-pad Seal Evaluation:* The gripper pads are projected onto the local point cloud surface and divided into 8 equal angular slices (45° each) to ensure comprehensive perimeter checking. Points are only considered if their vertical deviation falls within the *max sealing distance*, which models the physical deformation limit of the suction cup bellows. The algorithm calculates the global

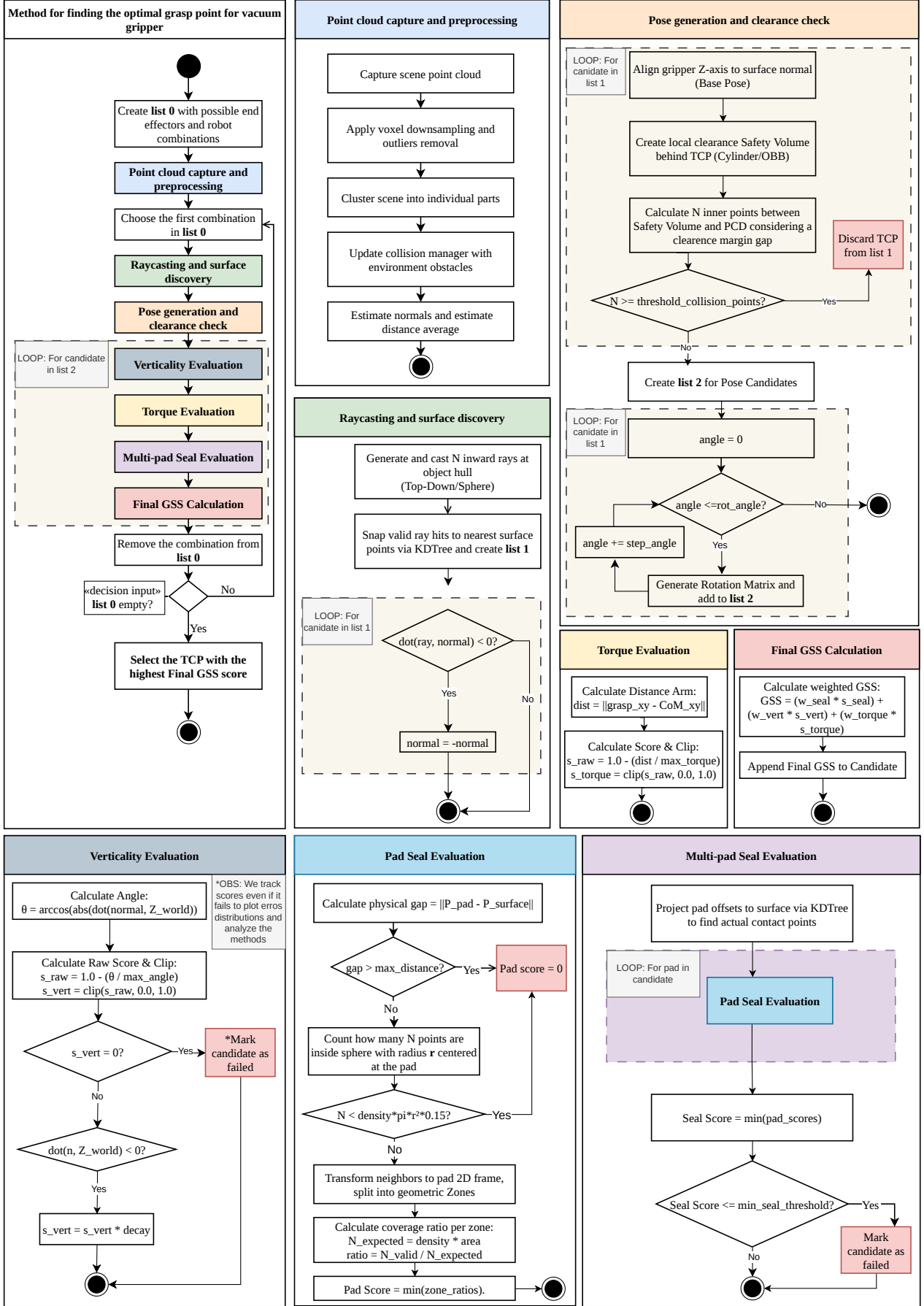


Figure 1: Overview of the proposed vacuum grasping algorithm pipeline, detailing the four main phases from point cloud preprocessing to the final target selection.

point cloud density based on average nearest-neighbor spacing, establishing an expected point count per slice. Candidates failing to meet a minimum density threshold (0.30) in any of the 8 slices are discarded to prevent vacuum leakage.

4. **Final Target Selection:** The surviving candidates are ranked by combining their normalized verticality, torque, and seal scores through a weighted sum. The candidate with the highest overall GSS is selected as the optimal grasp pose and forwarded to the robot’s motion planner for execution.

To ensure robustness across diverse object scales and sensor resolutions, the algorithm intentionally minimizes the use of hard-coded thresholds. Instead, it relies on dynamically computed parameters constrained by physical minimum and maximum bounds. For example, key geometric properties, such as the global point cloud density, are calculated at runtime based on the average nearest-neighbor point distance. This scale-invariant architecture allows the pipeline to process varying geometries, from small components like CubeSats to large logistics packages, without requiring manual parameter tuning for each new object.

Category	Parameter	Symbol / Value	Description / Calculation
<i>Dynamic Scale-Invariant Parameters</i>			
Evaluation	Global Point Density	$\rho = 1/d_{avg}^2$	Derived from average nearest-neighbor distance (d_{avg}).
Evaluation	Expected Pad Points	$N_{exp} = \rho \times A_{pad}$	Adapts seal expectation to current cloud resolution.
<i>Kinematic & Physical Constraints</i>			
Preprocessing	Normal Est. Neighbors	$k = 15$	Local points used for PCA surface normal estimation.
Generation	Approach Distance	100 mm	Safe standoff distance for the linear approach phase.
Generation	Collision Margin	0.05 (5%)	Volume expansion factor to absorb sensor/calibration noise.
Generation	Rotation Range	$\theta_{range} = 180^\circ$	Search space bounds adapted to gripper hardware symmetry.
Generation	Rotation Angle Step	$\theta_{step} = 45^\circ$	Angular resolution for generating Z-axis orientations.
Evaluation	Angular Seal Slices	$N_{slices} = 8$	Number of radial sectors evaluated per suction cup.
Evaluation	Max Sealing Distance	5 mm	Vertical tolerance modeling the physical suction cup bellow.
Evaluation	Maximum Tilt Angle	$\theta_{max} = 45^\circ$	Maximum allowed deviation from the gravity vector.
Evaluation	Upside-down Decay	0.60	Penalty factor applied to grasp poses pointing upwards.
Evaluation	Seal Min. Threshold	$S_{min} = 0.30$	Minimum acceptable point coverage ratio per pad zone.
<i>GSS Weighting Factors</i>			
Scoring	Verticality Weight	$w_{vert} = 0.40$	Importance of aligning with the gravity vector.
Scoring	Seal Weight	$w_{seal} = 0.30$	Importance of physical suction coverage.
Scoring	Torque Weight	$w_{torque} = 0.30$	Importance of minimizing planar distance to the CoM.

Table 1: Key Algorithmic Parameters and Dynamic Thresholds

2 Implementation and Software Framework

This section details the software architecture and the algorithmic implementation of the vacuum grasping pipeline. The framework is implemented in Python and follows a modular design to decouple geometric processing from hardware-specific parameters.

2.1 Software Architecture and Configuration Management

The grasping pipeline is implemented in Python, utilizing an object-oriented programming (OOP) architecture. This approach separates the geometric processing, collision evaluation, and hardware definition into distinct software modules. To manage data structures and algorithm states, the system relies on Python `dataclasses` combined with static type hinting. This design enforces strict structuring of input parameters and minimizes type-related runtime errors.

To ensure adaptability across different industrial assembly tasks and object geometries, the framework avoids hard-coded numerical values in the source code. Pipeline parameterization is handled through external YAML configuration files. Operational variables, such as collision margins, voxel grid resolutions, and Grasp Stability Score (GSS) weights, are imported dynamically during system initialization.

This parametric modularity extends to the geometric configuration of the end-effectors. The physical properties of the vacuum grippers are abstracted within dedicated hardware modules. Parameters including the number of suction cups, dynamic sealing radii, and relative distances between pad centers are defined externally. As a result, the core algorithmic class (`VacuumGraspSampler`) remains hardware-agnostic. Transitioning between evaluations using a single-cup gripper and an orthogonal four-cup configuration requires only the substitution of the respective configuration file, leaving the raycasting target discovery and evaluation logic unmodified.

2.2 Core Libraries and Computational Ecosystem

The computational efficiency of the framework depends on a specialized ecosystem of open-source Python libraries. Each library is selected to handle specific domains of the grasping pipeline, minimizing execution time and memory overhead.

Open3D: Open3D serves as the primary engine for 3D spatial data processing. While the pipeline is scripted in Python, Open3D operates on an optimized C++ backend, allowing the framework to process dense point clouds efficiently. Key functionalities include voxel downsampling, KD-Tree spatial indexing for nearest-neighbor queries, and tensor processing. Specifically, the surface discovery phase utilizes Open3D’s tensor-based `RaycastingScene`. This module enables the parallelized computation of thousands of inward-facing approach vectors simultaneously, bypassing the computational bottlenecks of standard Python operations.

NumPy: Linear algebra and mathematical evaluations within the Grasp Stability Score (GSS) loop are executed using NumPy. This library computes homogeneous transformations, coordinate frame rotations, and vector operations such as dot and cross products. By vectorizing the mathematical logic required for the torque and verticality evaluations, NumPy eliminates the need for iterative Python loops, significantly reducing the computational time required to process and rank candidate poses.

Trimesh: Geometric collision avoidance is managed by the `trimesh` library. While Open3D handles the environmental point cloud data, `trimesh` is strictly dedicated to

loading, inflating, and querying the physical 3D CAD meshes of the robotic end-effectors. It provides the boolean intersection algorithms required to verify spatial clearances between the inflated gripper volume and the target object, ensuring collision-free approach trajectories prior to hardware execution.

2.3 Point Cloud Ingestion and Preprocessing

The first stage of the computational pipeline involves standardizing the input 3D data. Raw point clouds generated from CAD assemblies or captured by depth sensors are frequently encoded in millimeters. To ensure dimensional compatibility with standard robotic simulation environments and kinematic solvers (e.g., ROS and MoveIt), the software applies a uniform scaling factor of 0.001 relative to the global origin (0,0,0) when the scale of the object is in millimeters. This operation explicitly converts all spatial coordinates to meters before any geometric evaluation occurs.

Following spatial normalization, the point cloud undergoes a voxel downsampling process. This algorithm divides the 3D space into a regular grid of volumetric elements (voxels) and replaces all points within each voxel with their geometric centroid. This operation serves two critical functions. First, it reduces the memory footprint and the total number of spatial queries required during the raycasting phase. Second, it normalizes the spatial density of the point cloud, compensating for areas of high concentration typical of high-resolution 3D scanners. A uniform global point density is essential to ensure that the density-dependent algorithms—specifically the multi-pad seal verification—perform consistently regardless of the initial sensor resolution.

Finally, a radius-based outlier removal filter is applied to the downsampled cloud. This filter removes isolated spatial noise, which is commonly introduced by depth sensor inaccuracies or reflective object surfaces. The algorithm evaluates the local neighborhood of each point and discards those that fail to meet a minimum threshold of neighbors within a specified radius. This filtering process provides a clean structural representation of the target object, preserving the sharp edges and flat planar surfaces strictly required for successful vacuum grasping.

2.4 Raycasting Engine and Hardware Symmetry

The target discovery phase avoids heuristic random sampling by employing a deterministic inward raycasting strategy. This mechanism is implemented using Open3D’s tensor-based `RaycastingScene` module. Approach vectors are formulated as tensor arrays and cast in batches toward the target object. By utilizing tensor data structures, the software achieves highly parallelized computation of surface intersections, avoiding the computational bottlenecks associated with iterative loops in standard Python.

Because the mathematically computed ray intersections do not inherently correspond to the discrete spatial coordinates of the input point cloud, a spatial mapping step is required. The software utilizes a `KDTree` nearest-neighbor search to snap each valid ray intersection coordinate to the closest exact physical point on the object’s surface. This ensures that the generated grasp candidates are grounded on actual sensor data rather than interpolated empty space.

After establishing a base approach vector aligned with the local surface normal, the algorithm generates a set of candidate orientations by rotating the gripper around this local Z-axis. To minimize computational overhead, this rotational search space is dynamically

constrained based on the geometric symmetry of the designated hardware configuration. For instance, a single circular suction cup possesses continuous radial symmetry, allowing the algorithm to bypass Z-axis rotation generation entirely (0°). Conversely, a bilateral two-pad gripper is evaluated over a bounded 180° range, while an orthogonal four-pad configuration is constrained to a 90° search space. This hardware-aware optimization reduces the total number of evaluated candidates, lowering processing time without compromising the discovery of valid grasp poses.

2.5 Collision Detection Engine

Before computing the Grasp Stability Score (GSS), the framework evaluates physical clearances to guarantee collision-free approach trajectories. This spatial verification is implemented using the `trimesh` library. The physical CAD mesh of the vacuum gripper is loaded and dynamically inflated by a predefined collision margin. This volumetric expansion acts as a safety buffer to absorb point cloud noise and mechanical calibration inaccuracies. A boolean intersection query is then executed between this inflated gripper mesh—positioned at the candidate approach pose—and the target object’s point cloud. Candidate poses that trigger an intersection are immediately rejected.

2.6 Vectorized Grasp Stability Evaluation (GSS)

After the geometric and collision constraints are satisfied, the remaining candidate poses are evaluated using the Grasp Stability Score (GSS). To ensure high-performance execution within the pipeline, the mathematical scoring functions are heavily vectorized using NumPy, reducing the evaluation time per candidate to fractions of a millisecond.

The first two components of the GSS are the Verticality and Torque evaluations. The Verticality score assesses the angular deviation of the approach vector relative to the global gravity vector (the Z -axis). This is computed efficiently across all candidates simultaneously using vectorized dot products. Similarly, the Torque evaluation minimizes the induced mechanical moment by calculating the planar distance in the XY coordinate space from each candidate’s Target Center Point (TCP) to the object’s estimated Center of Mass (CoM).

The third and most critical component is the Multi-pad Seal Verification, which algorithmically simulates the physical deformation limits of the suction cup bellows. For each candidate pose, the conceptual centers of the gripper pads are projected onto the local point cloud surface. A KD-Tree spatial index performs a rapid radial search to extract all neighboring surface points within the physical radius of each pad.

To guarantee a hermetic seal, it is insufficient to simply count the total number of points within this radius, as the surface might be discontinuous. Instead, the extracted points are partitioned into 8 distinct angular slices of 45° each, distributed radially around the local Z -axis of the pad. The algorithm then cross-references the point count in each slice against the expected dynamic point density (ρ) calculated during the preprocessing phase. If any of the 8 slices fails to meet the minimum point density threshold, the candidate pose is immediately rejected. This rigorous topological verification prevents vacuum leakage over holes, edges, or sparse surface geometries.